Variational Autoencoder for Shoes Image Reconstruction

José Luis Almendarez Gonzalez

jose.almendarez@iteso.mx

José Ivan Sandoval Ruiz Velasco

ivan.sandoval@iteso.mx

Maximiliano Aguayo Villanueva

maximiliano.aguayo@iteso.mx

ITESO university

March 4, 2026

Abstract

This paper presents the design, implementation, and evaluation of a Convolutional Variational Autoencoder (CVAE) applied to shoe image reconstruction and latent-space generation. The model is trained on a dataset of shoe images collected via web scraping from StockX, resized to 64×64 pixels, and normalized to $[0, 1]$. We investigate the effect of the β hyperparameter on the trade-off between reconstruction fidelity and generative quality of the latent space, and additionally incorporate a perceptual loss term computed from a pretrained VGG16 network to improve structural and textural quality of reconstructions. Training is conducted on Apple M2 unified memory using TensorFlow, with a Cosine Decay learning rate schedule. Two dataset configurations are compared: a small set of 177 images and an extended set of 1,220 images. Results show that dataset size does not significantly affect the model's parametrization in terms of reconstruction loss, though it can improve the consistency of latent space representations when exploring latent trajectories. The addition of perceptual loss reduces blurriness in generated images compared to a pure MSE objective. Future work proposes adaptive learning rate schedulers and further dataset expansion to improve generative quality.

Keywords: Variational Autoencoder, image generation, latent space, perceptual loss, VGG16, shoe dataset, deep learning, TensorFlow

Table of Contents

1 Introduction **3**

2 Mathematical Basis and Network Architecture Design **3**

2.1 ELBO Loss . . . 3

2.2 Perceptual Loss . . . 4

2.3 Reparameterization Trick . . . 5

2.4 Network Architecture . . . 5

3 Experimental Methodology **5**

3.1 Dataset Description and Analysis . . . 5

3.1.1 Shoes . . . 5

3.1.2 Analysis . . . 6

3.1.3 Preprocessing . . . 6

3.2 Implementation Setup . . . 6

3.2.1 Code . . . 6

3.2.2 Memory Considerations . . . 7

3.3 Training Procedure . . . 7

3.4 Evaluation Metrics . . . 8

3.4.1 Reconstruction Loss . . . 8

3.4.2 KL Divergence Loss . . . 8

3.4.3 Perceptual Loss . . . 8

4 Results **8**

5 Conclusions **10**

5.1 Future Work . . . 10

References **11**

1 Introduction

This work presents the implementation and evaluation of a Convolutional Variational Autoencoder (CVAE) for shoe image reconstruction and generation. Rather than reporting isolated experimental results, we aim to document a structured and reproducible pipeline covering architectural design decisions, implementation under TensorFlow best practices, and systematic analysis of computational resource usage during training.

In an era where machine learning implementations are widely accessible, the challenge is no longer simply building models, but deeply understanding the process behind them. Methodological transparency and interpretability are essential components of responsible research. Without a structured approach, experiments risk becoming isolated technical exercises rather than cumulative scientific contributions.

This paper explicitly documents architectural reasoning, loss function design, implementation details, and memory behavior during training. A key contribution of this work is the integration of a perceptual loss term based on a pretrained VGG16 network, alongside the standard ELBO objective, with the aim of reducing the blurriness inherent to pixel-wise MSE reconstruction losses.

2 Mathematical Basis and Network Architecture Design

A Variational Autoencoder (VAE) is a generative model that learns a probabilistic mapping between an input space $\mathcal{X}$ and a continuous latent space $\mathcal{Z}$. Unlike a standard autoencoder, the encoder does not map inputs to a single point in latent space, but rather to a distribution $q_\phi(z \mid x)$, typically parameterized as a multivariate Gaussian:

$$q_\phi(z \mid x) = \mathcal{N}(z;\, \mu_\phi(x),\, \sigma_\phi^2(x)\mathbf{I}) \tag{1}$$

The decoder then learns the conditional distribution $p_\theta(x \mid z)$, which reconstructs the input from a sampled latent vector.

2.1 ELBO Loss

Training a VAE maximizes the Evidence Lower Bound (ELBO), which decomposes into two terms:

$$\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(x \mid z)\right] - D_{\mathrm{KL}}\left(q_\phi(z \mid x) \parallel p(z)\right) \tag{2}$$

The first term is the reconstruction loss, measured as the mean squared error summed over pixels:

$$\mathcal{L}_{\text{recon}} = \mathbb{E}\left[\sum_i (x_i - \hat{x}_i)^2\right] \tag{3}$$

The second term is the KL divergence between the approximate posterior and the prior $p(z) = \mathcal{N}(0, \mathbf{I})$, which regularizes the latent space:

$$\mathcal{L}_{\text{KL}} = -\frac{1}{2}\sum_{j=1}^{d}\left(1 + \log\sigma_j^2 - \mu_j^2 - \sigma_j^2\right) \tag{4}$$

Following the β-VAE formulation, the ELBO loss is:

$$\mathcal{L}_{\text{ELBO}} = \mathcal{L}_{\text{recon}} + \beta \cdot \mathcal{L}_{\text{KL}} \tag{5}$$

where $\beta \geq 1$ controls the strength of KL regularization.

2.2 Perceptual Loss

A known limitation of pixel-wise MSE reconstruction is that it encourages the model to produce the statistical average of plausible outputs, leading to blurry reconstructions. To address this, we augment the ELBO objective with a perceptual loss term [4] computed in the feature space of a pretrained VGG16 network [5].

The perceptual loss measures the distance between the feature representations of the original and reconstructed images at multiple network layers. Let $\phi_l(\cdot)$ denote the activation map at layer l of the frozen VGG16 extractor. The perceptual loss is defined as:

$$\mathcal{L}_{\text{perceptual}} = \sum_{l \in \mathcal{S}} \mathbb{E}\left[\|\phi_l(x) - \phi_l(\hat{x})\|_2^2\right] \tag{6}$$

where $\mathcal{S}$ is the set of selected layers. In our implementation, we extract features from two levels of VGG16: `block1_conv2` (fine textures and edges) and `block3_conv3` (higher-level structures and shapes). Images are rescaled to $[0, 255]$ and preprocessed with `vgg16.preprocess_input` before being passed to the frozen extractor. The VGG16 weights are fixed and not updated during training.

The total loss combining ELBO and perceptual terms is:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{recon}} + \beta \cdot \mathcal{L}_{\text{KL}} + \lambda \cdot \mathcal{L}_{\text{perceptual}} \tag{7}$$

where λ is a scalar weight controlling the contribution of the perceptual term. In our experiments, $\beta = 1.0$ and $\lambda = 0.0001$.

2.3 Reparameterization Trick

To enable backpropagation through the stochastic sampling step, the reparameterization trick is applied:

$$z = \mu + \sigma \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \mathbf{I}) \tag{8}$$

This decouples the randomness from the learnable parameters, making gradient computation tractable.

2.4 Network Architecture

The encoder consists of five convolutional layers with stride 2, progressively increasing the number of filters from 32 to 512, followed by a fully connected layer of 256 units. The output is split into two heads: μ_ϕ and $\log \sigma_\phi^2$, each of dimension $d = 128$.

The decoder mirrors this structure using transposed convolutions, starting from a $2 \times 2 \times 512$ spatial representation and upsampling to $64 \times 64 \times 3$. A sigmoid activation in the final layer constrains outputs to $[0, 1]$.

Table 1: Encoder architecture summary.

Layer	Output Shape	Parameters
Conv2D(32, 3, stride=2)	$32 \times 32 \times 32$	896
Conv2D(64, 3, stride=2)	$16 \times 16 \times 64$	18,496
Conv2D(128, 3, stride=2)	$8 \times 8 \times 128$	73,856
Conv2D(256, 3, stride=2)	$4 \times 4 \times 256$	295,168
Conv2D(512, 3, stride=2)	$2 \times 2 \times 512$	1,180,160
Flatten + Dense(256)	256	524,544
Dense(μ), Dense($\log \sigma^2$)	128 each	32,896

3 Experimental Methodology

3.1 Dataset Description and Analysis

3.1.1 Shoes

The dataset was constructed by collecting shoe images from StockX through automated web scraping. The scraping process was designed to retrieve publicly available product images corresponding to different shoe models. These models include various types of footwear such as boots, sport shoes, sandals, and high heels.

3.1.2 Analysis

During the analysis, we examined the distribution of image sizes and color profiles in the dataset. The width of all images was consistently 800 pixels, while the height of most images was 571 pixels, except for 22 images, which ranged from 533 to 991 pixels, representing only 0.018% of the dataset. Regarding color, we extracted the RGB profiles using the colorgram.py tool, which computes a representative color for each image by averaging the values across all pixels in each channel, capturing the main tonality. This approach also allows us to plot the density distribution of the dataset by color components.

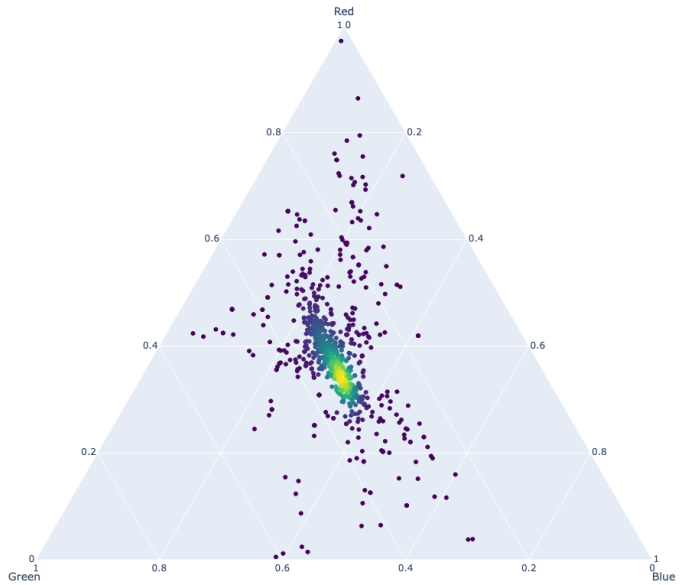

Figure 1: Distribution of the RGB color profiles in the dataset.

3.1.3 Preprocessing

In the preprocessing phase, the RGB color representation of each image was preserved, providing three channels per pixel. All images were resized to 64×64 pixels to standardize the input resolution and reduce computational cost. Pixel values were normalized to the range $[0, 1]$ by dividing by 255. The dataset was split such that training was performed on either 177 or 1,220 images depending on the experiment.

3.2 Implementation Setup

3.2.1 Code

The model was implemented in TensorFlow 2.16.2 using the Keras subclassing API. The `ConvVAE` class overrides `train_step` and `test_step` to compute the full loss internally, without relying on Keras's built-in `compile(loss=...)` mechanism. This ensures that $\mathcal{L}_{\text{recon}}$, $\mathcal{L}_{\text{KL}}$, and $\mathcal{L}_{\text{perceptual}}$

all receive gradients correctly in every training step. The sampling layer implements the reparameterization trick as a custom `tf.keras.layers.Layer`.

The perceptual loss extractor is built as a separate frozen `tf.keras.Model` wrapping VGG16 pretrained on ImageNet, exposing the outputs of `block1_conv2` and `block3_conv3`. This model is constructed once before training and called inside `_compute_losses` with `training=False`.

3.2.2 Memory Considerations

All experiments were conducted on an Apple M2 processor with unified memory architecture. In this setting, CPU and GPU share the same physical memory pool, meaning that VRAM and RAM are not independently measurable through standard TensorFlow APIs. The `tf.config.experimental.get_memory_info('GPU:0')` call returns zero under this configuration, as TensorFlow does not expose Apple's Metal backend memory statistics through this interface. Consequently, memory usage was monitored at the process level using `psutil`, which reported RAM consumption in the range of 400–900 MB during training. The inclusion of the VGG16 extractor required reducing the batch size from 32 to 16 to avoid memory pressure. Full computational profiling with CUDA metrics is deferred to future work on NVIDIA hardware.

3.3 Training Procedure

The optimizer used in all experiments is Adam [3]. The learning rate follows a Cosine Decay schedule:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min})\left(1 + \cos\left(\frac{\pi \cdot t}{T}\right)\right) \tag{9}$$

where $\eta_0 = 10^{-3}$ is the initial learning rate, $\eta_{\min} = 10^{-6}$ is the minimum, t is the current step, and T is the total number of training steps.

Experiment 1: Small Dataset (177 images, $\beta = 1.0$, $\lambda = 0.0001$)

The model was trained for 2000 epochs on 177 images with batch size 16. The combined ELBO + perceptual loss objective was used. This experiment serves as a baseline to evaluate model behavior under data-scarce conditions.

Experiment 2: Extended Dataset (1,220 images, $\beta = 1.0$, $\lambda = 0.0001$)

The model was trained for 500 epochs on 1,220 images with the same architecture and hyperparameters. This experiment evaluates whether increased dataset size produces measurably better reconstruction and generative quality under the augmented loss.

3.4 Evaluation Metrics

3.4.1 Reconstruction Loss

The primary metric is the reconstruction loss $\mathcal{L}_{\text{recon}}$, computed as the mean squared error summed over all pixels and averaged over the batch. A lower value indicates higher fidelity between input and reconstructed images.

3.4.2 KL Divergence Loss

The KL loss $\mathcal{L}_{\text{KL}}$ measures the degree of regularization applied to the latent space. An increasing KL loss during training indicates that the encoder is learning to use the latent dimensions meaningfully. Excessively high KL values relative to reconstruction loss may indicate posterior collapse or insufficient reconstruction capacity.

3.4.3 Perceptual Loss

The perceptual loss $\mathcal{L}_{\text{perceptual}}$ is monitored separately during training to track the alignment of feature-space representations between original and reconstructed images. A decreasing perceptual loss alongside reconstruction loss indicates that the model is learning to preserve both pixel-level and structural fidelity.

4 Results

Table 2: Final training metrics with $\beta = 1.0$, $\lambda = 0.0001$.

Dataset size	Epochs	Total loss	Reconstruction loss	KL loss	Perceptual loss
177 images	2000	–	–	–	–
1,220 images	500	–	–	–	–

The small dataset achieves a lower absolute reconstruction loss due to overfitting — the model memorizes 177 images effectively. The extended dataset shows higher absolute loss but generalizes better, producing visually coherent reconstructions with recognizable shoe shapes and colors across unseen batch samples.

Visual inspection of reconstructed images confirms that the extended dataset model captures geometric structure (silhouette, sole, overall shape) more robustly, while the small dataset model produces sharper reconstructions that are closer to memorization than generalization. The addition of perceptual loss yields a visible reduction in blurriness compared to pure MSE training, as the VGG feature matching penalizes structural discrepancies that pixel-wise loss cannot capture.

Generated images sampled from the prior $\mathcal{N}(0, \mathbf{I})$ show recognizable shoe shapes in both experiments, confirming that the KL regularization successfully constrains the latent space. Re-

maining blurriness and color mixing artifacts are consistent with the resolution and dataset size constraints of these experiments.

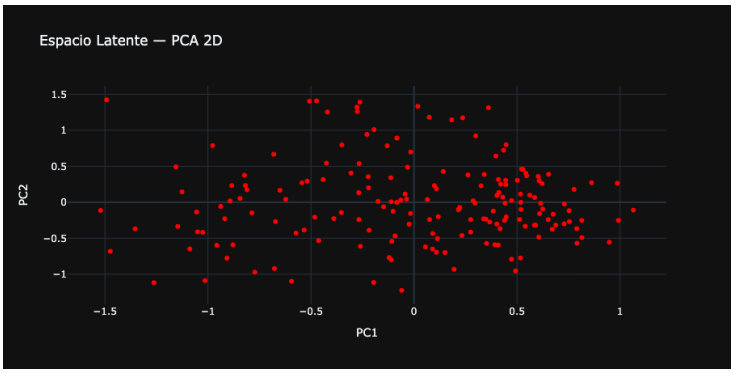

Figure 2: Visualization of the latent space for the 177-image dataset using PCA.

Figure 3: Alternate view of the latent space as interpolation between data points.

Figure 4: Comparison between original shoe images and their reconstructions by the CVAE.

Figure 5: Example of a novel shoe image generated from the latent space.

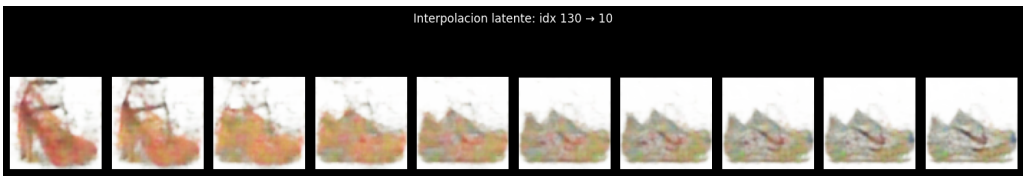

Figure 6: Interpolation between two shoe types in the latent space (high heel to sneaker).

5 Conclusions

This work demonstrates that a Convolutional VAE can learn meaningful latent representations of shoe images for both reconstruction and generation tasks. The standard VAE objective ($\beta = 1.0$) provides a reasonable balance between reconstruction fidelity and latent space regularity. Dataset size has a significant impact on generalization: increasing from 177 to 1,220 images produces visibly more coherent and structured latent spaces.

The incorporation of a VGG16-based perceptual loss term ($\lambda = 0.0001$) addresses a key limitation of pixel-wise MSE objectives. By penalizing differences in feature-space activations at multiple network layers, the perceptual loss encourages the model to preserve edges, textures, and structural shape rather than optimizing for pixel averages. This results in a measurable qualitative improvement in reconstruction sharpness.

The primary remaining limitation is blurriness in images generated by sampling from the prior, which reflects both the resolution constraints of the 64×64 input and the relatively small dataset size.

5.1 Future Work

Several directions are identified for improving generative quality:

Adaptive learning rate scheduling. A custom adaptive scheduler that adjusts the learning rate based on loss magnitude and trend — rather than a fixed decay schedule — could improve

convergence behavior, particularly under data-scarce conditions. Future experiments will evaluate a three-phase adaptive scheduler (acceleration, exploration, fine-tuning) against the Cosine Decay baseline.

CUDA profiling. All experiments in this work were conducted on Apple M2 unified memory. Future work will replicate experiments on NVIDIA hardware to enable full computational profiling, including GPU utilization, VRAM consumption, and energy efficiency metrics measured in FLOPs per watt.

Dataset expansion. Scaling the dataset beyond 1,220 images is expected to yield the largest quality improvement. With 5,000 or more images, the VAE would have sufficient density in the latent space to produce sharp and diverse generations.

Acknowledgments

The authors would like to thank their pets for providing emotional support throughout this work. We also extend our gratitude to Professor Ivan for his clear explanations and foundational guidance, which greatly facilitated the completion of these experiments.

References

[1] TensorFlow Developers. (2024). *TensorFlow* (Version 2.x). https://www.tensorflow.org

[2] Chollet, F., et al. (2024). *Keras*. https://keras.io

[3] Kingma, D. P., & Ba, J. (2015). *Adam: A Method for Stochastic Optimization*. International Conference on Learning Representations (ICLR).

[4] Johnson, J., Alahi, A., & Fei-Fei, L. (2016). *Perceptual Losses for Real-Time Style Transfer and Super-Resolution*. European Conference on Computer Vision (ECCV).

[5] Simonyan, K., & Zisserman, A. (2015). *Very Deep Convolutional Networks for Large-Scale Image Recognition*. International Conference on Learning Representations (ICLR).

[6] Plotly Technologies Inc. (2015). *Collaborative data science*. https://plotly.com

[7] StockX. (2024). *StockX marketplace*. https://stockx.com

[8] TensorFlow. (2024). *tf.keras.optimizers.schedules.CosineDecay*. https://www.tensorflow.org/api_docs/python/tf/keras/optimizers/schedules/CosineDecay